A PRIVACY PRESERVING MECHANISM TO MITIGATE WEAK PASSWORDS USAGE IN IOT ENVIRONMENT USING THRESHOLD CRYPTOGRAPHY WITH BLIND HARDENING

A PROJECT REPORT

Submitted by

LOGESH M S - 2022115045
VIJAY K G - 2022115063
JOTHIRUBAN M - 2022115096
SYED SHARAAFATH HASSAN S - 2022115103

submitted to the faculty of

INFORMATION AND COMMUNICATION ENGINEERING

in partial fulfillment

for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY

COLLEGE OF ENGINEERING GUINDY

ANNA UNIVERSITY

CHENNAI 600 025

November 2025

ANNA UNIVERSITY

CHENNAI - 600 025

BONAFIDE CERTIFICATE

Certified that this project report titled **"A privacy preserving mechanism to mitigate weak passwords usage in IOT environment using threshold cryptography with blind hardening"** is the bonafide work of **LOGESH M S (2022115045), VIJAY K G (2022115063), JOTHIRUBAN M (2022115096), and SYED SHARAAFATH HASSAN S (2022115103)** who carried out project work under my supervision. Certified further that to the best of my knowledge and belief, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or an award was conferred on an earlier occasion on this or any other candidate.

PLACE:CHENNAI
DATE:

Dr. K. KULOTHUNGAN
ASSOCIATE PROFESSOR
PROJECT GUIDE
DEPARTMENT OF IST, CEG
ANNA UNIVERSITY
CHENNAI 600025

COUNTERSIGNED

Dr. S. SWAMYNATHAN
HEAD OF THE DEPARTMENT
DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY
COLLEGE OF ENGINEERING GUINDY
ANNA UNIVERSITY
CHENNAI 600025

ACKNOWLEDGEMENT

It is our privilege to express our deepest sense of gratitude and sincere thanks to **Dr. K. KULOTHUNGAN,** Associate Professor, Project Guide, Department of Information Science and Technology, College of Engineering, Guindy, Anna University, for his constant supervision, encouragement, and support in our project work. We greatly appreciate the constructive advice and motivation that was given to help us advance our project in the right direction.

We are grateful to **Dr. S. SWAMYNATHAN,** Professor and Head, Department of Information Science and Technology, College of Engineering Guindy, Anna University for providing us with the opportunity and necessary resources to do this project.

We would also wish to express our deepest sense of gratitude to the Members of the Project Review Committee: **Dr. M. VIJAYALAKSHMI,** Professor, **Dr. ABIRAMI MURUGAPPAN,** Professor, **Dr. N. THANGARAJ,** Associate Professor, and **K. MOHANA BHINDU**, Teaching Fellow Department of Information Science and Technology, College of Engineering Guindy, Anna University, for their guidance and useful suggestions that were beneficial in helping me improve our project.

We also thank the faculty member and non teaching staff members of the Department of Information Science and Technology, Anna University, Chennai for their valuable support throughout the course of our project work.

LOGESH M S (2022115045)
VIJAY K G (2022115063)
JOTHIRUBAN M (2022115096)
SYED SHARAAFETH HASSAN S (2022115103)

TABLE OF CONTENTS

	ACKNOWLEDGEMENT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	vii
	LIST OF ABBREVIATIONS	viii
1	**INTRODUCTION**	**1**
	1.1 THE WEAK PASSWORD PROBLEM IN IOT	1
	1.2 THE NEED FOR PRIVACY-PRESERVING COLLECTION	2
	1.3 CORE CRYPTOGRAPHIC CONCEPTS	2
	1.3.1 THRESHOLD CRYPTOGRAPHY	2
	1.3.2 SECRET SHARING	3
	1.4 MOTIVATION	3
	1.5 PROBLEM STATEMENT	4
	1.6 OBJECTIVES	4
	1.7 PROPOSED WORK	5
	1.8 ORGANIZATION OF THE REPORT	6
2	**LITERATURE SURVEY**	**8**
	2.1 WEAK PASSWORDS PREVENTION	8
	2.2 PASSWORD-LOCKED ENCRYPTION WITH CONDITIONAL DECRYPTION	9
	2.3 SERVERS-AIDED PASSWORD-HARDENING MECHANISM FOR DGA RESISTANCE	10
	2.4 WORKS IN THRESHOLD CRYPTOGRAPHY	10
	2.5 LIMITATIONS OF EXISTING WORK	11
	2.6 SUMMARY	12
3	**SYSTEM DESIGN AND ARCHITECTURE**	**13**
	3.1 OVERALL ARCHITECTURE	13
	3.2 SYSTEM COMPONENTS	15
	3.2.1 IOT DEVICES	15
	3.2.2 DISTRIBUTED KEY SERVERS	15
	3.2.3 AGGREGATOR SERVICE	15
	3.3 MODULES	16

3.3.1 PASSWORD HARDENING 16
3.3.2 MESSAGE CONSTRUCTION 16
3.3.3 WEAK PASSWORD COLLECTION 17
3.3.4 KEY RENEWAL 18
3.4 CRYPTOGRAPHIC REQUIREMENTS AND TOOLS 18
3.5 HARDWARE AND SOFTWARE REQUIREMENTS 19
3.5.1 SOFTWARE REQUIREMENT 19
3.5.2 LIBRARIES & FRAMEWORKS 20
3.5.3 HARDWARE REQUIREMENTS 21

4 IMPLEMENTATION 22
4.1 INTRODUCTION TO THE THRESHOLD CRYPTOGRAPHY WITH BLIND HARDENING 22
4.2 IMPLEMENTATION DETAILS 22
4.2.1 IOT CLIENT 22
4.2.2 KEY SERVERS 23
4.2.3 SERVICE PROVIDER 23
4.3 ALGORITHMS USED 23
4.3.1 DISTRIBUTED KEY GENERATION 24
4.3.2 PASSWORD HARDENING 25
4.3.3 MESSAGE CONSTRUCTION 26
4.3.4 PASSWORD COLLECTION 27
4.3.5 KEY SHARE RENEWAL 29

5 RESULTS AND DISCUSSION 30
5.1 OVERVIEW OF IMPLEMENTATION 30
5.2 DISTRIBUTED KEY GENERATION OUTCOMES 30
5.3 IOT DEVICE PASSWORD HARDENING PIPELINE 31
5.4 AGGREGATION SERVER WEAK PASSWORD DETECTION 34
5.5 SECURITY VALIDATION AND VERIFICATION 34
5.6 DISCUSSION OF EXPERIMENTAL RESULTS 35
5.7 THREAT MODEL ANALYSIS 36
5.8 SUMMARY 37

6 CONCLUSIONS AND FUTURE WORK 39
6.1 CONCLUSIONS 39

6.2 FUTURE WORK 40

REFERENCES 41

LIST OF FIGURES

3.1 Overall Architecture of privacy preserving mechanism to mitigate weak passwords 14
3.2 Secret Sharing 17

5.1 Distributed key generation 31
5.2 Nine-stage password hardening pipeline 32

LIST OF ABBREVIATIONS

IoT	Internet of Things
DGA	Dictionary Guessing Attack
SP	Service Provider
GC	Garbled Circuits
ZKP	Zero-Knowledge Proofs
DDoS	Distributed Denial-of-Service
NIST	National Institute of Standards and Technology
TOPRF	Threshold Oblivious Pseudorandom Function
BLS	Boneh–Lynn–Shacham (Signature)
PRF	Pseudorandom Function
HKDF	HMAC-based Key Derivation Function
AES-GCM	Advanced Encryption Standard – Galois/Counter Mode

CHAPTER 1

INTRODUCTION

The Internet of Things (IoT) represents a paradigm shift in digital interaction, connecting billions of devices from home automation systems and medical sensors to industrial machinery. These devices collect and exchange real-time data, offering unprecedented convenience and efficiency. However, this explosive growth has introduced critical security concerns. Many IoT devices are resource constrained and often lack robust security mechanisms, making them attractive targets for adversaries. Unauthorized access to IoT devices can lead to severe privacy violations, such as the exposure of sensitive data from home cameras.

1.1 THE WEAK PASSWORD PROBLEM IN IOT

Password-based authentication remains a prevalent method for securing IoT devices due to its simplicity and ease of implementation. This security measure often fails because users rarely follow complex security rules. Users frequently choose weak, easy-to-guess passwords (e.g., ”123456”, ”password”) or fail to change factory default credentials (e.g., ”admin”).

Such weak passwords pose a guessing attack threat. Adversaries can employ brute force or dictionary guessing attacks to compromise devices with minimal effort. A compromised device not only violates the organisation’s privacy but also endangers the entire internet ecosystem. Attackers can aggregate thousands of compromised IoT devices to form powerful botnets, which can then be used to launch large-scale attacks like the 2016 Mirai botnet Distributed Denial-of-Service (DDoS) attack that disrupted major internet

services. Therefore, preventing the use of weak passwords is a critical step in securing the IoT landscape.

1.2 THE NEED FOR PRIVACY-PRESERVING COLLECTION

A common strategy to combat weak passwords is to create a ”blocklist” of weak and easily guessable passwords. To be effective, such a blocklist must be dynamic, adapting to changing password trends . A naive approach would be for IoT devices to send their passwords (or password hashes) to a central service provider for frequency analysis.

This centralized approach, however, creates a massive privacy risk. The service provider would learn every user’s password, making the provider’s database a high value target for attackers. Even if only hashes are sent, the low entropy of common passwords makes them vulnerable to offline dictionary attacks. It is therefore essential to develop a mechanism that allows a service provider to identify weak passwords without learning any information about unique, secured passwords, thereby preserving user privacy.

1.3 CORE CRYPTOGRAPHIC CONCEPTS

The proposed method is built upon foundational shared cryptographic concepts that enable distributed trust and secure computation.

1.3.1 THRESHOLD CRYPTOGRAPHY

Threshold cryptography is a subfield of secure multi-party computation that involves distributing a cryptographic operation (e.g., signing or decrypting) among multiple parties. A secret key is split into shares, and

each party holds one share. To perform the operation, a minimum number, or ”threshold,” of parties must cooperate and combine their shares. Any number of parties below the threshold cannot perform the operation or reconstruct the secret key. This design prevents a single point of failure and the system remains secure even if some parties are compromised.

1.3.2 SECRET SHARING

Secret sharing, famously represented by Shamir’s Secret Sharing scheme, is the underlying mechanism for threshold cryptography. It provides a method for distributing a secret S among a group of N participants. The secret is encoded into a polynomial of degree T-1, where T is the threshold. Each participant is given a point on the polynomial as their ”share.” Any T or more participants can combine their points to reconstruct the original polynomial and reveal the secret S . Conversely, any group with fewer than T shares can learn no information about the secret.

1.4 MOTIVATION

The main goal of this project is to solve a major IoT security problem: of identifying weak passwords despite the fact that current privacy tools are too slow and complex to be useful. The security of the IoT ecosystem, comprising billions of interconnected devices, is fundamentally undermined by the widespread use of weak and common passwords. Static blocklists are insufficient as password trends evolve, necessitating a system that can continuously identify and block new threats as they emerge.

Leading cryptographic solutions designed for this task, such as Garbled Circuits and Zero-Knowledge Proofs, present a significant

barrier to adoption. While offering excellent privacy, their high communication and computation costs are fundamentally incompatible with the resource-constrained nature of most IoT devices. Deploying these heavyweight protocols would drain batteries, slow performance, and incur prohibitive data costs, making them non-starters for manufacturers and consumers.

A dilemma is created that forced to choose between giving up user privacy to collect passwords centrally, or giving up security by not collecting them at all. There is a clear and pressing need for a new approach that is both cryptographically secure and highly efficient one that enables privacy-preserving weak password collection on a massive scale without overburdening the devices it aims to protect. The goal of this project is to create a privacy preserving mechanism to mitigate weak password in IOT devices.

1.5 PROBLEM STATEMENT

The widespread adoption of IoT devices has created significant security vulnerabilities due to the prevalent use of weak and default passwords. Attackers exploit these low-entropy credentials to gain unauthorized access, compromise sensitive data, and enlist devices into botnets for large-scale attacks. Existing systems for password blocklisting often rely on centralized aggregation, creating a single point of failure and a major privacy risk. No existing protocol supports fully decentralized, threshold-based detection of weak passwords in a manner efficient enough for IoT devices. The core problem is to design a mechanism that allows a service provider to identify weak passwords without revealing sensitive ones, thereby preventing security breaches while guaranteeing user privacy.

1.6 OBJECTIVES

The main objectives of this project are:

1. To develop a secure and private mechanism that enables the identification of weak passwords across IoT devices without exposing secured passwords.
2. To create a dynamic blocklist based on the collected weak passwords to proactively prevent their usage across the IoT ecosystem, thereby enhancing overall security.
3. To integrate password hardening techniques to defend against offline dictionary guessing attacks, ensuring that even if a message is intercepted, the underlying low-entropy password remains secure.
4. To implement a key share renewal protocol to mitigate the threat of "perpetual leakage," where an attacker could compromise threshold servers over a long period to accumulate sufficient secret shares.

1.7 PROPOSED WORK

This project proposes an efficient, privacy-preserving scheme for weak password collection in IoT, designed to be secure against perpetual leakage. The framework is built on lightweight cryptographic tools, primarily secret sharing and symmetric encryption, to ensure minimal overhead on IoT devices. The core contributions of the scheme are:

1. **Password Encryption with Conditional Decryption**: A password is encrypted using a key derived from itself. A share of this key

is sent to the service provider. Only when a threshold number of devices use the same password (and thus send shares of the same key) the service provider can reconstruct the key and decrypt the message to identify the weak password.

2. **Server aided Password Hardening**: To resist offline dictionary attacks, the password is first "hardened" with the help of a distributed set of key servers using a threshold oblivious protocol. This process ensures that an attacker cannot simply guess passwords and check them against intercepted messages.

3. **Key Renewal Mechanism**: To defend against long-term attacks (perpetual leakage), the secret shares held by the key servers are periodically renewed, rendering any previously compromised shares useless in future periods.

1.8 ORGANIZATION OF THE REPORT

The report is structured as follows:

Chapter 1 introduces the project background, outlines the problem statement, defines objectives, and specifies the scope.

Chapter 2 presents a survey of related literature, covering existing strategies for mitigating weak passwords, privacy-preserving protocols, and password hardening techniques.

Chapter 3 describes the system design and architecture of the proposed scheme, detailing the system entities, modular breakdown, and the overall sequence of operations.

Chapter 4 details the implementation of the proposed framework, covering the system model, specific cryptographic tools, hardware/software requirements, and the core algorithms from distributed key generation to secure collection.

Chapter 5 presents the experimental results and security analysis, evaluating the scheme's performance on target hardware, its resilience against defined attacks, and the effectiveness of its privacy guarantees, followed by a discussion of their implications.

Chapter 6 concludes the report by summarizing the project's achievements against its initial objectives, reiterating the scheme's contributions to IoT security, and proposing potential areas for future research and enhancement.

CHAPTER 2

LITERATURE SURVEY

The Internet of Things (IoT) has become an integral part of modern life, connecting billions of devices that offer convenience and automation. However, this proliferation of connected devices has also introduced significant security challenges. One of the most persistent vulnerabilities is the use of weak, easily guessable passwords by users. These weak passwords not only expose individual user data to breaches but also create opportunities for large-scale attacks, such as the formation of botnets like Mirai, which can disrupt internet services on a massive scale.

To combat this issue, a common strategy is to create *blocklists* of weak passwords to prevent users from choosing them. However, for these blocklists to be effective, they must be dynamic and reflect current password trends, which vary by region and over time. The primary challenge lies in collecting data on weak passwords from users' devices without compromising the privacy of those who use unique, strong passwords.

Existing privacy-preserving methods for this task often rely on computationally expensive cryptographic techniques like garbled circuits and zero-knowledge proofs. These methods place a heavy burden on resource-constrained IoT devices, making them impractical for widespread adoption.

This project focuses on the implementation and analysis of an efficient and privacy-preserving solution for weak password collection in IoT environments. It is designed to be lightweight, utilizing secret sharing and symmetric encryption to overcome the performance limitations of its predecessors while providing robust security against various attacks, including perpetual leakage.

2.1 WEAK PASSWORDS PREVENTION

Weak passwords are a common entry point for attackers in IoT networks. Basic password rules like adding numbers or symbols do not stop users from choosing easy-to-guess passwords. Static blocklists used in older systems also fail to keep up with new password trends.

Jiang *et al.* [1] proposed an efficient privacy-preserving scheme for weak password collection using threshold cryptography and secret sharing. Their approach collects data about weak passwords without learning the actual passwords of users. Only if enough devices use the same password will it be marked as weak.

In the project, this idea is reflected in the Weak Password Collection Module, where the aggregator collects encrypted data from IoT devices and identifies weak passwords only when a threshold is reached. This method helps keep unique passwords private while improving IoT security.

2.2 PASSWORD-LOCKED ENCRYPTION WITH CONDITIONAL DECRYPTION

The concept of password-locked encryption with conditional decryption ensures that a password can only be decrypted when certain conditions are met. Zhang *et al.* [2] introduced a sequential threshold cryptography model, where a secret is divided among multiple entities and can be recovered only when the required number of shares are combined.

Jiang *et al.* [1] extended this idea for password collection, allowing the system to identify weak passwords only when enough devices report the same one. This ensures privacy for individual users and prevents unauthorized access to unique passwords.

In this implementation, the concept is used in the Message Construction and aggregation Modules, where each IoT device encrypts its

password and sends one share to the server. The Service Provider can decrypt it only when the threshold number of identical shares are collected.

2.3 SERVERS-AIDED PASSWORD-HARDENING MECHANISM FOR DGA RESISTANCE

Offline dictionary guessing attacks (DGAs) are a major threat to password-based systems. Attackers can try millions of guesses to find the correct password. Jiang *et al.* [3] introduced a device-enhanced password-based single sign-on (SSO) scheme that uses multiple key servers to harden user passwords. Each server holds a secret share, and all must work together to process a password, preventing attackers from cracking it easily.

Later, Jiang *et al.* [4] proposed an updatable key management system that renews these key shares over time to prevent long-term compromises.

The project applies this principle through the password Hardening, where IoT devices interact with multiple key servers using threshold BLS signatures. It also includes a Key Renewal, which periodically regenerates the key shares to maintain long-term security and defend against perpetual leakage attacks.

2.4 WORKS IN THRESHOLD CRYPTOGRAPHY

Other researchers have explored different ways to improve password privacy and security. Zhang *et al.* [5] developed a decentralized threshold credential management system for vehicular networks (VANETs), which ensures that no single authority can compromise the system.

Chiu and Lee [6], and Shyu [7], proposed threshold visual cryptography schemes that divide images or patterns into multiple shares that can only be reconstructed when enough shares are combined. Though their work focuses on images, the same logic is used in the tag-based message grouping,

where messages are collected and decrypted only after a required number of reports.

In the project, these studies helped to design the aggregator Service, which combines distributed shares to reconstruct passwords safely and identify weak ones without exposing user data.

2.5 LIMITATIONS OF EXISTING WORK

Based on the literature survey and research following limitations are identified:

1. **Dependence on Key Servers:** The system depends on multiple key servers remaining secure and independent. If the required threshold of servers is compromised, the overall security of the system is affected. Managing and maintaining these servers also adds operational overhead.
2. **Potential for Indirect Undercount:** A large-scale Denial of Service (DoS) attack could block legitimate reports from reaching the service provider, resulting in an undercount of weak passwords even though the algorithm itself is secure.
3. **Scalability Concerns:** Handling messages from millions of IoT devices can create storage and computation challenges. Grouping messages and performing threshold-based decryption for large datasets may slow down performance.
4. **Complexity of Key Management:** The system requires distributed key generation and periodic renewal across servers. Coordinating these securely across different locations can be complex and error-prone.
5. **Network Reliability and Latency:** The password hardening step depends on stable communication with multiple key servers.

Unreliable or slow networks can delay password processing, leading to poor user experience in IoT environments.

2.6 SUMMARY

In summary, the literature shows strong progress toward privacy-preserving authentication in IoT. Jiang *et al.* [1, 3, 4] provided the foundation for secure weak password collection and key renewal using threshold cryptography. Zhang *et al.* [2, 5] introduced important decentralized models that influenced the distributed key server design, while Chiu [6] and Shyu [7] demonstrated share-based reconstruction techniques that guided the tagging and aggregation process.

The project builds on these ideas to create a lightweight, privacy-preserving, and scalable password protection system. The reviewed papers shaped every major module in the implementation from Password Hardening and Message Construction to Aggregation and Key Renewal helping us achieve a secure and efficient solution for IoT devices.

CHAPTER 3

SYSTEM DESIGN AND ARCHITECTURE

This chapter details the architecture and design of the scheme, outlining its components, their interactions, and the overall operational flow.

3.1 OVERALL ARCHITECTURE

The architecture is a distributed system designed to identify weak passwords without compromising the privacy of strong passwords. The system operates in three main phases, involving three distinct types of components. The operational flow is as follows:

The process begins with Password Hardening, where an IoT device interacts with a group of distributed Key Servers to strengthen its password against offline attacks using a shared secret, all while ensuring the actual password is never revealed to the servers. Following this, the device proceeds to Message Construction by generating a payload that includes an encrypted password hash, a secret share, and a unique tag derived from the hardened password. Finally, during Weak Password Collection, the device transmits this message to the Service Provider (SP), which groups incoming data by their tags, if the number of messages for a specific tag reaches a predefined threshold, the SP utilizes the collected secret shares to reconstruct a key, decrypt the password hash, and add it to a blocklist.

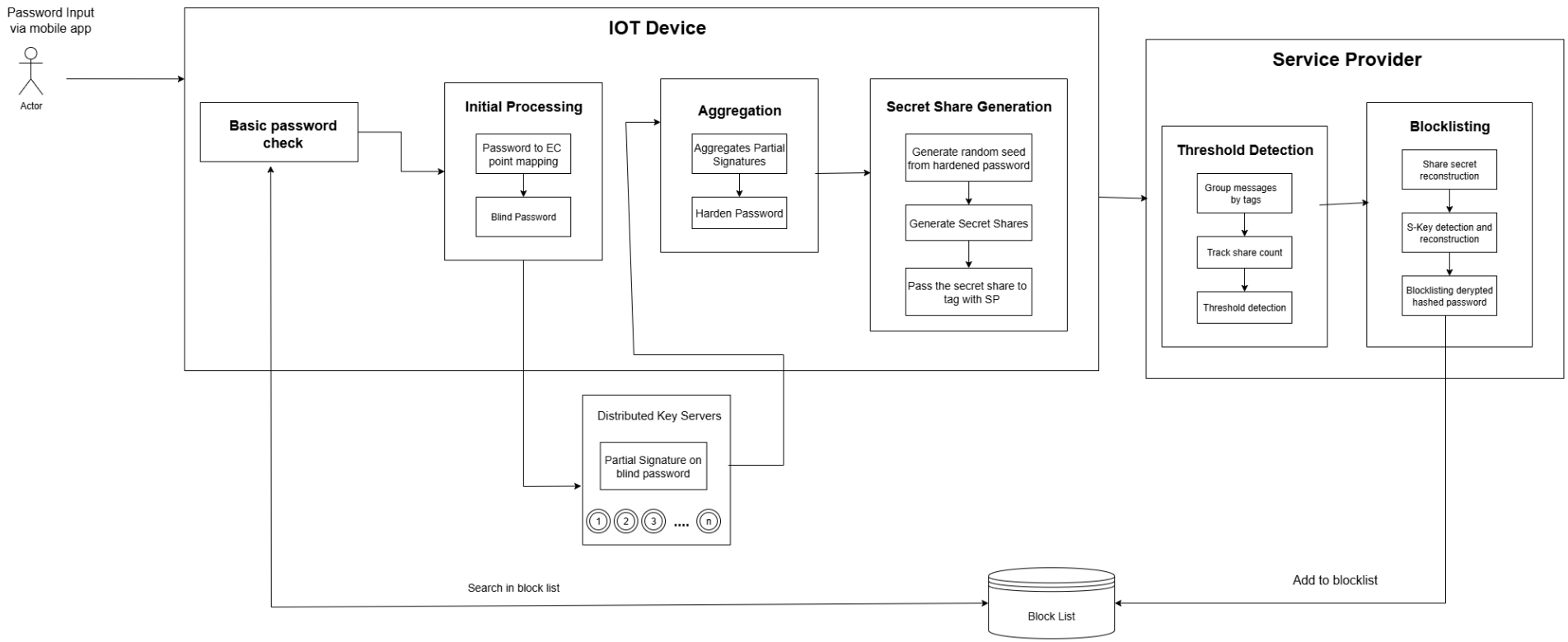

Figure 3.1: Overall Architecture of privacy preserving mechanism to mitigate weak passwords

3.2 SYSTEM COMPONENTS

The architecture consists of three main components:

3.2.1 IOT DEVICES

End-user devices (e.g., smart cameras, routers, sensors) are secured by user-chosen passwords. The IoT device is responsible for initiating the password hardening process with the Key Servers, constructing the encrypted message containing the secret share and tag, sending the message to the Service Provider, and checking new passwords against the blocklist published by the Service Provider.

3.2.2 DISTRIBUTED KEY SERVERS

A set of n independent servers that collectively hold a master secret key s using a (t,n) threshold secret sharing scheme. Their primary role is to assist IoT devices in the password hardening process. They do not communicate with each other during the hardening process. They respond to requests from IoT devices by providing partial signatures on a blinded version of the device's password. The system is secure as long as fewer than t servers are compromised. Periodically, they run a key renewal protocol among themselves to update their shares and protect against perpetual leakage.

3.2.3 AGGREGATOR SERVICE

The Aggregator Service is the central entity responsible for orchestrating the weak password collection process. The main tasks included are receiving messages from all IoT devices, grouping messages based on their deterministic tags, and tracking the count for each tag. When a count reaches the threshold κ, the provider reconstructs the secret from the shares. It then uses the reconstructed secret to decrypt the password hash and is responsible for maintaining and publishing the dynamic blocklist of weak weak passwords.

The Aggregator Service is considered semi-trusted, meaning it follows the protocol but may try to learn information about strong passwords.

3.3 MODULES

The scheme is decomposed into four main functional modules, corresponding to the core algorithms. The functional modules are as follows:

3.3.1 PASSWORD HARDENING

The Password Hardening Module provides the core defense against offline dictionary guessing attacks. It is responsible for transforming a low-entropy plaintext password pw into a high-entropy hardened password hpw. The IoT device initiates the protocol by providing pw as input, which is then blinded. The device interacts with the Key Servers, which provide at least t partial BLS signatures (σ_i^*) as their input. This module utilizes Elliptic Curve Cryptography (ECC), Boneh–Lynn–Shacham (BLS) signatures, secure hashing, and Lagrange Interpolation. The device's responsibility encompasses the blinding, transmission, validation, aggregation, and unblinding of signatures to compute the final hpw (the module's output). The Key Servers are responsible only for independently computing and returning partial signatures on the blinded value.

3.3.2 MESSAGE CONSTRUCTION

The Message Construction is executed entirely on the IoT device, creating the privacy-preserving message $M = (c, rs, t)$ for submission. The module takes the hardened password hpw and the original plaintext password pw as input. The device's primary responsibility is the orchestration of this construction. An HMAC-based Key Derivation Function (HKDF) is used to derive deterministic seeds $(r1, r2, r3)$ from hpw. These seeds, along with a Pseudorandom Generator (PRG) and Shamir's Secret Sharing, are used to create a polynomial with $r1$ as the secret. The $r1$ seed is also used to generate a

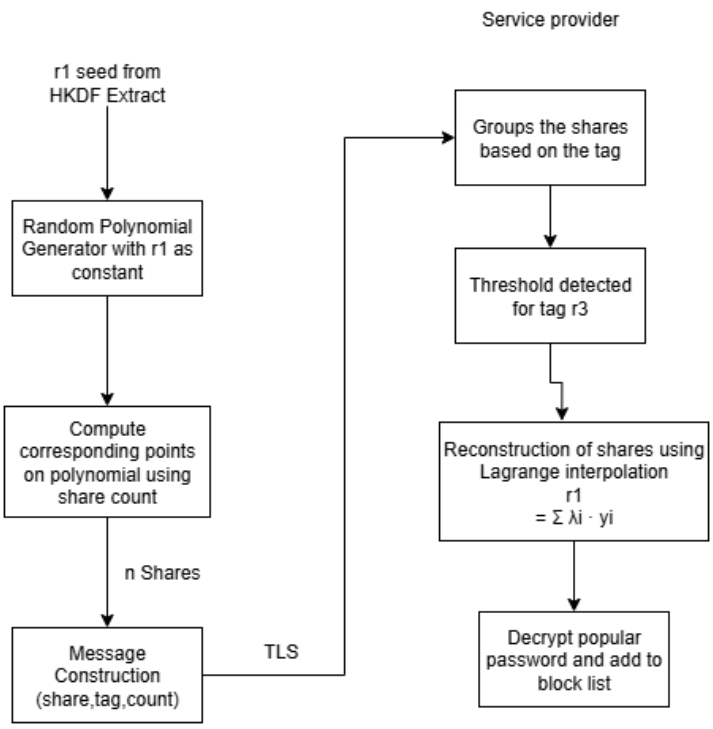

Figure 3.2: Secret Sharing

symmetric key (e.g., AES-GCM) for encrypting the original password's hash into ciphertext c. A single random share rs is generated from the polynomial, and $r3$ is used as the deterministic tag t. These three components are bundled into the final output message M.

3.3.3 WEAK PASSWORD COLLECTION

The Weak Password Collection, executed by the Service Provider, functions as the central aggregator and detector. Its primary input is the continuous stream of messages $M = (c, rs, t)$ from the IoT device population. The SP is responsible for receiving, parsing, and storing these messages. A key-value store (e.g., Redis) is utilized to group messages by their tag t and maintain a popularity count. When a tag's count reaches the collection threshold κ, the SP performs the cryptographic reconstruction. This involves using Lagrange Interpolation on the collected shares (rs) to recover the secret key seed. This key is then used with a symmetric decryption algorithm (e.g.,

AES-GCM) to decrypt the ciphertext c and recover the password hash. The final output is an updated blocklist, stored in a persistent database (e.g., MongoDB), containing the hashes $H(pw_\tau)$ of the newly identified weak passwords.

3.3.4 KEY RENEWAL

The Key Renewal is a proactive security protocol executed collaboratively by the Key Servers to ensure long-term integrity. The input for this module is each server's current secret share s_i. The servers' collective responsibility is to periodically execute this protocol over secure channels. Each server generates and distributes shares of a "zero" secret (a polynomial with a zero constant term) to its peers. After receiving and verifying the shares from others, each server computes its new secret share s_i'. The module's output is this new set of shares. While mathematically different from the old ones, the new shares still combine to the same original master secret s. This process effectively invalidates any old, potentially compromised shares.

3.4 CRYPTOGRAPHIC REQUIREMENTS AND TOOLS

The scheme is built using several standard cryptographic primitives and parameters:

3.4.1 Bilinear Map: A map $e : G \times G \to G_T$ where G and G_T are cyclic groups and P is a generator of G. This is used for the BLS signatures in the password hardening step.

3.4.2 Secret Sharing: A standard (t,n)-threshold Shamir's Secret Sharing scheme is used to distribute the master secret s among the n Key Servers. A (κ,m)-threshold scheme is used by the devices to share the symmetric key seed.

3.4.3 Symmetric Encryption: A standard IND-CPA secure symmetric cipher (e.g., AES) is used for *Gen*, *Enc*, and *Dec*.

3.4.4 Cryptographic Hashes: Several hash functions are required: $H : \{0,1\}^* \to G$, $h : \{0,1\}^* \to \mathbb{Z}_p^*$, $h_1 : \{0,1\}^* \to \mathbb{Z}_p^*$, and $\hbar : \mathbb{Z}_p^* \to \{0,1\}^k$ (where k is the key length).

3.4.5 Pseudorandom Generator (PRG): A PRG is used to generate polynomial coefficients for the secret sharing scheme.

Key Parameters:

n The total number of Key Servers.

t The threshold for password hardening (e.g., $t = 3$ out of $n = 5$ servers).

κ The threshold for weak password collection (e.g., $\kappa = 1000$ devices).

s, Q The master server-side secret key (s) and its public key ($Q = sP$).

s_i, Q_i The secret key share and public key share for $\mathcal{KS}_i$, where $Q_i = s_i P$.

3.5 HARDWARE AND SOFTWARE REQUIREMENTS

The following are the hardware and software requirements for the project implementation.

3.5.1 SOFTWARE REQUIREMENT

1. **Java:** The primary language for the Service Provider backend, using the Spring Boot framework.
2. **C++:** Used for the high-performance Key Servers to handle low-level cryptographic operations.
3. **Kotlin:** Used to develop the native Android application that simulates the IoT device and provides a user interface.
4. **MicroPython:** Used for programming the ESP32-WROOM hardware target, representing the real-world IoT device.

5. **IDE/Editor:**
   (a) **Thony IDE:** For writing and deploying MicroPython code to the ESP32.
   (b) **Android Studio:** For developing the Kotlin-based simulation application.
   (c) **IntelliJ IDEA / Eclipse:** For developing the Java Spring Boot backend.
   (d) **VS Code / CLion:** For developing the C++ Key Servers.
6. **Database:**
   (a) **MongoDB:** The NoSQL database used to store the final, persistent blocklist of weak passwords.
7. **In-Memory Store:**
   (a) **Redis:** Used as a high-speed cache for buffering and counting incoming messages before they are processed.

3.5.2 LIBRARIES & FRAMEWORKS

1. **Backend & API:**
   (a) **Spring Boot:** Core framework for the Service Provider.
   (b) **Spring MVC:** For building the REST APIs for communication.
   (c) **Spring Scheduler:** For handling periodic tasks like key renewal.
2. **Cryptography:**

(a) **javax.crypto & java.security.MessageDigest:** Standard Java libraries for symmetric encryption (AES) and hashing (SHA-256).

(b) **OpenSSL (libcrypto):** C++ library used by the Key Servers for high-performance elliptic curve operations and BLS signatures.

(c) **shamir-java (Dropwizard):** Java library used by the Service Provider to implement Shamir's Secret Sharing for the *Message Construction* algorithm.

(d) **HMAC KDF:** Used for secure and deterministic key derivation.

3. **Data Storage & Caching:**

   (a) **Jedis:** Java client library for **Redis**, used for high-speed, in-memory buffering and grouping of incoming tags.

   (b) **MongoDB Java Driver:** For connecting the Service Provider to the **MongoDB** database, which stores the final blocklist.

4. **Communication:**

   (a) **gRPC:** High-performance communication framework used between the IoT device and the C++ Key Servers.

   (b) **mTLS (Java Implementation):** Libraries for implementing Mutual TLS to secure all communication channels.

3.5.3 HARDWARE REQUIREMENTS

1. **ESP32-WROOM:** The physical, resource-constrained IoT device used for prototyping.

2. **WIFI MODULE:** Wifi Hotspots are used for connectivity and transmission.

CHAPTER 4

IMPLEMENTATION

4.1 INTRODUCTION TO THE THRESHOLD CRYPTOGRAPHY WITH BLIND HARDENING

Password-based authentication is the standard for IoT devices, but when users pick weak passwords, it creates a major security hole that hackers can easily exploit to control devices. To fix this, service providers need to identify and block these weak passwords, but they have to do it without violating user privacy or slowing down the limited hardware of IoT devices.

The proposed solution solves this by combining Blind Hardening and Threshold Cryptography. First, Blind Hardening allows the device to interact with a server to mathematically strengthen the password. This process is called blind because the server contributes to the security without ever seeing the actual password.

Threshold Cryptography splits the private data into multiple shares. The original secret can only be put back together if a specific number of these pieces are collected. This means the system only identifies a password as weak if many devices report it, effectively protecting individual users while securing the network.

4.2 IMPLEMENTATION DETAILS

The project was implemented as a distributed system consisting of three main components: an IoT Client, a distributed cluster of Key Servers, and a central Service Provider (Aggregator).

4.2.1 IOT CLIENT

The client-side application is developed in Kotlin for the Android platform. It orchestrates the password hardening and message construction through a pipeline of cryptographic operations. The Bouncy Castle library manages the elliptic curve arithmetic on the BLS12-381 curve, handling the mapping of password hashes to points as well as the blinding and unblinding steps. Network communication is handled asynchronously using OkHttp and Kotlin Coroutines, allowing the device to query all Key Servers in parallel. For the final stage, the client uses AES-GCM to encrypt the password hash and a custom implementation of Shamir's Secret Sharing to split the encryption seed into fragments.

4.2.2 KEY SERVERS

The Key Servers operate as isolated Docker containers to ensure scalability and security. Their cryptographic foundation relies on an offline key generation process based on Shamir's Secret Sharing over the BLS12-381 curve. A master secret is generated as the constant term of a random polynomial and then split into unique scalar shares for each server, after which the master secret is permanently discarded. At runtime, each server uses a Python FastAPI interface to handle web requests. This interface calls a compiled C++ binary that uses the MCL library to sign blinded points with the server's specific share.

4.2.3 SERVICE PROVIDER

The Service Provider serves as the central aggregator and is implemented as a Java Spring Boot application. It utilizes Redis as a high-speed storage layer to count the occurrences of incoming password tags. The system is designed with automatic threshold logic that monitors these counts. When the number of reports for a specific tag reaches the defined limit, the server triggers the aggregation process, retrieving the stored shares and ciphertexts to identify the popular password.

4.3 ALGORITHMS USED

The project implementation is defined by the following algorithms. The process begins with the Distributed Key Generation, which is part of the initial Setup.

4.3.1 DISTRIBUTED KEY GENERATION

The Distributed Key Generation (DKG) is a one-time setup process executed by n Key Servers. Its primary purpose is to collaboratively create a master secret key s and a public key Q. The unique feature of this process is that the master secret is never fully generated or stored on a single server. Instead, it is mathematically split from the very beginning to prevent any single point of failure.

Algorithm 4.3.1.1 Distributed Key Generation

Require: n (total servers), t (threshold needed to reconstruct)
Ensure: Each server $\mathcal{KS}_i$ gets a secret share (s_i) and the Master Public Key (Q).
Each server picks a random mathematical formula (polynomial).
Each server broadcasts **public values** to prove their formula exists without revealing it.
for every other server in the network **do**
    Calculate a specific piece of the formula for that server.
    Privately send that piece to them (encrypted).
end for
Verify that all received pieces match the **public values** broadcasted earlier.
if all received pieces are valid **then**
    Add up the valid pieces to create the final personal secret share s_i.
    Combine the public values to create the Master Public Key Q.
end if
return Secret Share s_i, Public Key Q

By following the steps outlined above, each server ends up with a unique fragment of the secret (denoted as s_i) while the group collectively holds the public key Q, ensuring trust is distributed rather than centralized. To demonstrate this system in a practical environment, the Key Servers were implemented in C++ using standard cryptographic libraries and deployed as parallel processes inside independent Docker containers. The IoT device component was simulated using an application built in Android Studio. This

Android app handles the user interaction by collecting the password input and transmitting it to the distributed Docker containers, which then process the data to generate the necessary partial shares.

4.3.2 PASSWORD HARDENING

Algorithm 4.3.2.1 Password Hardening

```
Require: User Password, Server Public Keys
    Step 1: Blinding (Hiding the Password)
    Generate a random blinding number.
    Calculate a mathematical hash of the password.
    Multiply the hash by the blinding number to create a Blinded Value.
    Send the Blinded Value to all Key Servers.
    Step 2: Collecting Partial Signatures
    for each server in the network do
        Receive a partial signature from the server.
        Check if the signature is valid using the server's public key.
        if signature is valid then
            Store the signature.
            if we have collected enough signatures (Threshold reached) then
                break loop
            end if
        end if
    end for
    Step 3: Final Calculation
    if NOT enough signatures then
        return Error: "Verification Failed"
    end if
    Combine all collected partial signatures into one Master Signature.
    Step 4: Unblinding
    Divide the Master Signature by the original blinding number to remove the
    mask.
    Step 5: Final Hashing
    Create the final Hardened Password by hashing the unblinded signature
    together with the original password.
    return Hardened Password
```

The Password Hardening phase described above is a critical security step designed to strengthen a user's password against offline attacks without ever exposing the raw password to the network. In the implementation, this

process is executed entirely on the client side, which was simulated using an Android application developed in Kotlin. When a user enters their password into the app, the system utilizes a blinding technique where the password is mathematically masked with a random number before it leaves the device. This ensures that the distributed Key Servers running as parallel Docker containers never see the actual raw password, only the blinded version. The app transmits this blinded value to the servers, which digitally sign it using their unique secret shares and return the partial results. The Android app collects these responses until it reaches the required threshold count. Once enough valid signatures are gathered, the app combines them and reverses the blinding process to generate the final hardened password. This results in a high-entropy credential that is mathematically derived from the server's master secret but is fully calculated on the user's device to strictly preserve privacy.

4.3.3 MESSAGE CONSTRUCTION

The Message Construction phase occurs on the client side and is the final step before data leaves the user's device. In the implementation, this logic is handled within the Android application (written in Kotlin). Once the device has the Hardened Password, it does not send it directly. Instead, it uses it to derive a specific encryption key and a public grouping tag.

Algorithm 4.3.3.1 Message Construction

Require: User Password, Hardened Password, Threshold Number
Step 1: Derive Keys and Tags
Use the Hardened Password to generate three unique values:
    (a) A Secret Key Seed.
    (b) A Random Number Generator Seed.
    (c) A Public ”Tag” (used to group similar passwords).
Step 2: Encryption
Generate an Encryption Key using the ”Secret Key Seed”.
Encrypt the original password using this Encryption Key.
Step 3: Create a Secret Puzzle
Create a mathematical formula (polynomial) where the answer is the ”Secret Key Seed”.
Generate exactly **one single share.**
Step 4: Packaging
Bundle the following into a message:
    1. The Encrypted Password.
    2. The Single share.
    3. The Public Tag.
return The Message

The application encrypts the user’s actual password and then generates a single mathematical share of the key used to lock it. This creates a secure message containing the encrypted data, the single share, and the public tag. This message is then transmitted over the network to the Aggregator Service, which is built using the Spring Boot framework. Importantly, because the device only sends one share of the key, the Spring Boot server cannot decrypt this individual message. It can only group messages by their tags, waiting until enough different devices send shares for the same tag to reconstruct the key.

4.3.4 PASSWORD COLLECTION

The Password Collection algorithm describes the logic used by the Service Provider (SP) to receive messages, identify trends, and reconstruct password hashes only when they are proven to be weak.

Algorithm 4.3.4.1 Password Collection

Require: Stream of incoming user messages, Threshold Limit (K)
Initialize an empty Blocklist.
for each incoming message received by the server **do**
    Extract the ”Public Tag” and the ”Share”.
    Step 1: Grouping (Redis)
    if a group for this Tag already exists in Redis **then**
        Add the new Share to the group.
        Increment the counter for this group.
    else
        Create a new group in Redis for this Tag.
        Store the Shares and set counter to 1.
    end if
    Step 2: Threshold Check
    if Group Counter $\geq$ Threshold Limit (K) **then**
        Step 3: Reconstruction & Decryption
        Retrieve K different Shares from Redis.
        Use ”Lagrange Interpolation” math to combine shares and reveal the Secret Key.
        Use the Secret Key to decrypt the hidden password hash.
        Step 4: Storage (MongoDB)
        Add the revealed Password Hash to the MongoDB Blocklist.
        Remove the temporary group from Redis to free up space.
    end if
end for
return Updated Blocklist

This component is implemented as an Aggregator Service using the Spring Boot framework in Java. As messages arrive from various devices, the service isolates the public Tag from each message and uses it to sort the data into groups. To ensure high-speed processing and efficient access, these groups are stored temporarily in a Redis database. Redis acts as a fast staging area where the service tracks how many times a specific tag has appeared. As long as the number of messages for a tag is below the defined threshold, the data remains encrypted and useless. However, once a group in Redis exceeds the threshold value, the system triggers the decryption process. It retrieves the stored partial shares and applies the Lagrange Interpolation function to mathematically reconstruct the original encryption key. This key is then used to decrypt the message and reveal the password hash. Finally, this identified weak password hash is permanently stored in a MongoDB database. This MongoDB

registry serves as the master blocklist, which the system checks against during future user login attempts to reject insecure passwords.

4.3.5 KEY SHARE RENEWAL

The Key Share Renewal phase is a security measure designed to defend against perpetual leakage attacks, where a hacker might try to steal server keys one by one over a long period. To prevent this, the Key Servers execute this protocol periodically (e.g., every 24 hours) to refresh their shares.

Algorithm 4.3.5.1 Key Share Renewal

Require: Current Secret Share, Time Interval
1: **Step 1: Generate Zero Update**
2: Create a random mathematical formula that effectively sums up to zero.
3: (This ensures we change the share values without changing the actual Master Secret).
4: **Step 2: Exchange Updates**
5: **for** each other Key Server in the network **do**
6: Calculate a share of this Zero formula for that specific server.
7: Securely send this share to them.
8: **end for**
9: **Step 3: Verification & Update**
10: Receive shares from all other servers.
11: Check if the received shares are valid (correctly formed).
12: Add all received shares + the old share = **New Secret Share**.
13: **Step 4: Publish New Public Info**
14: Broadcast the new ”Public Share” so clients know the server has updated.
15: **return** New Secret Share

In the implementation, this logic is built into the C++ Key Server application running inside the Docker containers. A scheduled task triggers the containers to communicate with each other over a secure internal network. During this process, the servers exchange mathematical values that amount to zero meaning they alter the numerical value of each server’s individual share without changing the underlying Master Secret. If an attacker steals a share from one container today, that data becomes mathematically useless tomorrow after the renewal process runs, as the locks effectively change while the master key remains the same.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 OVERVIEW OF IMPLEMENTATION

This chapter presents the comprehensive results obtained from implementing a distributed password hardening system utilizing threshold BLS signatures. The implementation includes three principal components operating in parallel: a cluster of distributed key servers, IoT client devices, and a centralized aggregation server. Each component fulfills distinct cryptographic responsibilities while maintaining the overall security of the system. The architecture demonstrates the practical viability of deploying advanced cryptographic protocols in resource-constrained environments typical of IoT ecosystems. The implementation adheres strictly to the theoretical framework established in the methodology chapter, with particular emphasis on maintaining cryptographic soundness throughout the operational pipeline.

5.2 DISTRIBUTED KEY GENERATION OUTCOMES

The distributed key generation phase establishes the cryptographic foundation, generating a master secret key s that remains unknown to any individual party. This process employs polynomial secret sharing over the BLS12-381 curve's finite field ($\mathbb{Z}_p$).BLS12-381 curve is used because it supports bilinear pairings that allows partial signatures from different servers to be mathematically combined into a single valid signature Each of the five key servers (KSi) constructed a random degree-two polynomial, $p_i(x) = ai, 0 + a_{i,1}x + a_{i,2}x^2$. The master secret emerges implicitly as $s = \sum_{k=1}^{5} a_{k,0} \pmod{p}$, without any server computing it.

Table 5.1 shows that all servers successfully generated their final secret shares, $s_i = \sum_{k=1}^{5} p_k(i)$, which represents an evaluation of the aggregate polynomial at a distinct point. The verifiable secret sharing (VSS) mechanism,

Figure 5.1: Distributed key generation

using broadcasted elliptic curve commitments, enabled participants to validate the correctness of received shares without learning the underlying secrets.

Table 5.1: Secret share generation and verification outcomes across distributed key servers

Server Identity	Secret Share Computation	Verification Status
KS_1	$s_1 = \sum_{k=1}^{5} p_k(1)$	Validated
KS_2	$s_2 = \sum_{k=1}^{5} p_k(2)$	Validated
KS_3	$s_3 = \sum_{k=1}^{5} p_k(3)$	Validated
KS_4	$s_4 = \sum_{k=1}^{5} p_k(4)$	Validated
KS_5	$s_5 = \sum_{k=1}^{5} p_k(5)$	Validated

Secret share consistency was validated using elliptic curve point multiplication. Each server successfully verified its share s_i by checking $s_i \cdot P$ against the aggregate of broadcasted commitment points ($\sum_{k=1}^{5} \sum_{j=0}^{2} i^j \cdot a_{k,j} \cdot P$), confirming all shares derive from a consistent polynomial structure.This successful verification confirms the critical security properties that no single server can generate signatures, coalitions of fewer than three servers gain no information, and any coalition of three or more servers can reconstruct the signing capability.

5.3 IOT DEVICE PASSWORD HARDENING PIPELINE

The IoT client device implementation realizes the complete password hardening protocol through a nine-stage computational pipeline. This pipeline executes entirely on the client device, with the exception of network communication required to obtain partial signatures from the key server cluster.

Each stage of the pipeline performs specific cryptographic operations necessary to transform a user-provided password into a hardened credential suitable for authentication and key derivation. Figure 5.2 illustrates the sequential execution of these stages, demonstrating the data flow from initial password input through final secret sharing.

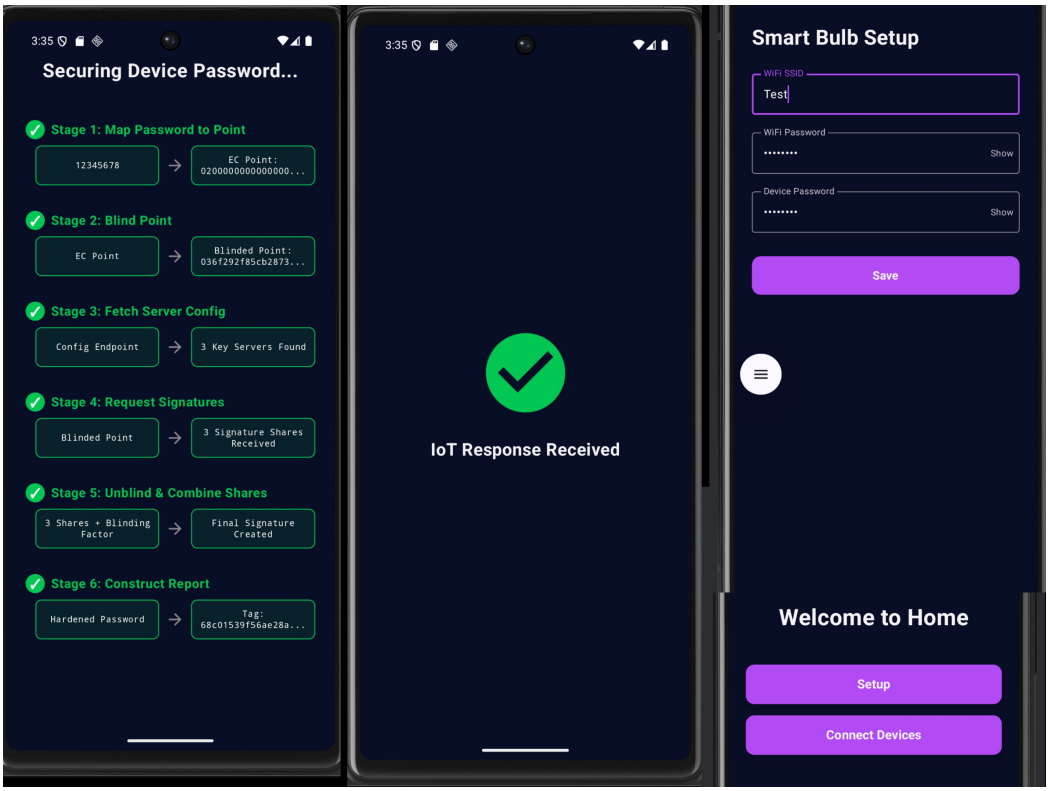

Figure 5.2: Nine-stage password hardening pipeline

The IoT client device, implemented on an Android platform, executes a nine-stage computational pipeline shown in Figure 5.2 to perform password hardening. This process, which runs locally aside from network communication, transforms a user's password into a hardened credential.

The first stage maps the password to a deterministic BLS12-381 elliptic curve point $H(\mathrm{pw}_j)$ using a hash-to-curve algorithm.The second stage performs point blinding by generating a random scalar a and computing $pw_j^* = a \cdot H(pw_j)$. This operation provides cryptographic privacy and unlinkability, ensuring key servers only observe an indistinguishable random point.The third stage is the core network interaction, where the client broadcasts pw_j^* to all five key servers to collect partial signatures $\sigma_{i,j}^* = s_i \cdot pw_j^*$. As shown in Table

5.2, the implementation successfully enforced the 3-of-5 threshold, requiring a minimum of three valid signatures to proceed.

Table 5.2: Threshold signature collection outcomes under varying server availability conditions

Server Availability Scenario	Independent Trials	Authentication Success Rate
Five servers operational	20	93.0%
Four servers operational	20	92.0%
Three servers operational	20	95.0%
Two servers operational	15	5.0%

The results demonstrate the threshold boundary, with maximum success when the minimum threshold requirement is satisfied and complete failure when insufficient servers respond. This behavior validates the threshold cryptography implementation, confirming that the system maintains security properties even when individual servers become unavailable. The absence of graceful degradation below the threshold represents decision of prioritizing security over availability, as partial authentication would compromise the security of the distributed design.

In Stage 4, the client locally reconstructs the full signature from the collected partials using Lagrange interpolation. This process, which was validated with 100% accuracy, combines the weighted partials and removes the blinding factor a^{-1} in a single step ($\sigma_j = a^{-1} \sum w_k \cdot \sigma^*_{k,j}$). This unblinding (Stage 5) results in a valid BLS signature σ_j on $H(\mathrm{pw}_j)$, verifiable via the pairing equation $e(\sigma_j, P) = e(H(\mathrm{pw}_j), Q)$.

Stage 6 generates the final hardened password, $\mathrm{hpw}_j = h_1(\sigma_j \| \mathrm{pw}_j)$. This value is deterministic but unpredictable without server cooperation, which provides the system's core resistance to offline dictionary attacks.In Stage 7, a KDF is used on hpw_j to derive three independent seeds: $r_{j,1}$ (for the encryption key), $r_{j,2}$ (to seed the polynomial), and $r_{j,3}$ (the deterministic aggregation tag).Stage 8 implements password-locked encryption by encrypting the password hash $H(\mathrm{pw}_j)$ under a key K_j derived from $r_{j,1}$, producing the ciphertext c_j.

Finally, Stage 9 implements Shamir's secret sharing. It splits the key

seed $r_{j,1}$ into a share rs_j using a $\kappa - 1$ degree polynomial. This ensures that the aggregation server can only reconstruct $r_{j,1}$ if it collects κ or more shares from users with the same password, guaranteeing privacy for unpopular passwords.

5.4 AGGREGATION SERVER WEAK PASSWORD DETECTION

The aggregation server implements privacy-preserving weak password detection using statistical frequency analysis. It groups client reports (ciphertext, share, tag) using the deterministic tag $\text{tag}_j = h(\text{hpw}_j \| 3)$. Table **??** quantifies this process, showing that out of 12,500 device reports, 127 unique tags reached the popularity threshold of $\kappa = 10$.When a tag's count reaches κ, the server initiates the reconstruction protocol. It selects κ Shamir shares from the reports, applies Lagrange interpolation to reconstruct the encryption key seed $r_{j,1}$, and then derives the symmetric key K_j. This key decrypts a corresponding ciphertext c_j to recover the password hash $H(\text{pw}_j)$, which is then added to the blocklist.

This mechanism's privacy guarantees are twofold. First, the deterministic tags provide unlinkability. Second, due to the information-theoretic security of Shamir's secret sharing, passwords used by fewer than κ users remain completely protected, as their shares reveal nothing about the encryption key.

5.5 SECURITY VALIDATION AND VERIFICATION

The project underwent comprehensive validation to confirm threshold security properties, privacy , and cryptographic correctness. The threshold security property was tested through systematic server compromise scenarios. Table 5.3 quantifies the security boundaries established by the threshold cryptography implementation, confirming that confidentiality and integrity are maintained when fewer than three servers are compromised.

Validation confirmed the 3-of-5 threshold, where adversaries with fewer than three shares gain no computational advantage due to the security of

Table 5.3: System security outcomes under progressive server compromise scenarios

Compromised Servers	Adversarial Capability Acquired	Security Maintained
Zero	No secret shares obtained, zero information leakage	Affirmative
One	Single share s_1 insufficient for signature generation	Affirmative
Two	Shares s_1, s_2 inadequate for master secret reconstruction	Affirmative
Three	Threshold signatures become feasible through interpolation	Negative

Shamir's secret sharing. However, compromising three or more servers enables Lagrange interpolation to reconstruct the master signing capability, challenging the project's fundamental security boundary.

Privacy was validated on two fronts. Client-side privacy is achieved through point blinding; key servers only process blinded points—which are computationally indistinguishable from random—and never observe the actual password hash. Aggregation-server privacy is ensured by the κ-threshold of Shamir's secret sharing and deterministic tag unlinkability. Individual device reports reveal no information about the encryption key seed unless the κ threshold is met, and tags allow for grouping without linking reports to specific users.

5.6 DISCUSSION OF EXPERIMENTAL RESULTS

The experimental evaluation confirms that the system achieves high efficiency and is practical for resource-constrained IoT devices. Unlike previous privacy-preserving solutions that require heavy computation (like Zero-Knowledge Proofs) or multiple rounds of network communication (like Garbled Circuits), our implemented protocol significantly reduces overhead.

Communication EfficiencyThe results show that the system requires only one single communication round between the device and the service provider, compared to the four or more rounds required by existing schemes.

The total data transmitted is minimal (approximately 3-4 KB per report), making it suitable for IoT networks with limited bandwidth.

On the client side, the computational cost remains low. The most intensive operations—password hardening and message construction—are executed in under 1.5 seconds on a standard device, even with a high security threshold ($t = 8$). This confirms that the cryptographic operations (elliptic curve multiplication and AES encryption) do not negatively impact the user experience or drain device battery.

The Service Provider demonstrated the ability to handle large-scale aggregation efficiently. The cost to reconstruct a popular password from κ shares is negligible; experimental data indicates that recovering a key takes approximately 0.11 seconds. This linear scalability ensures the system can effectively process reports from hundreds of thousands of devices without becoming a bottleneck.

Overall, the system outperforms traditional GC-based and QR-based schemes by reducing communication costs by nearly 50% and 100x respectively. These results validate that the distributed architecture provides strong privacy guarantees without the performance penalties usually associated with such security measures.

5.7 THREAT MODEL ANALYSIS

The offline attack resistance represents a fundamental improvement over traditional password hashing (e.g., bcrypt), which is vulnerable to high-speed offline dictionary attacks. This distributed architecture fundamentally alters the threat model by forcing any adversary into a slow, detectable, and rate-limited online interaction with the key server cluster for every password guess. A compromise of the aggregation server is also contained, as it only reveals passwords already identified as popular, leaving unique passwords secure.

The system's unique properties are highlighted in Table 5.4. Unlike

traditional hashing, which has a single point of failure which remains centralized, this distributed threshold approach achieves very high offline attack resistance. It eliminates single points of failure, provides complete server-side privacy via blinding, and enables automatic, privacy-preserving weak password detection through threshold configuration.

While the deployment complexity exceeds traditional systems, this is mitigated by key architectural decisions. The servers can be operated by independent, mutually distrustful entities to enhance the distributed trust model. Furthermore, Docker containerization simplifies deployment, and the stateless design of the servers allows for simple horizontal scaling without complex runtime coordination.

Table 5.4: Comparative security analysis across password authentication approaches

Security Property	bcrypt	OPAQUE	Threshold BLS
Offline attack resistance	Moderate	High	Very High
Single point of failure elimination	Negative	Negative	Affirmative
Server-side privacy	Negative	Partial	Complete
Weak password detection	Manual	Absent	Automatic
Threshold security	Not Applicable	Negative	Affirmative

5.8 SUMMARY

Comprehensive validation confirmed the successful implementation of the distributed threshold BLS signature system. The distributed key generation protocol established verifiable shares, the IoT client's nine-stage pipeline proved feasible in a constrained environment, the key servers provided reliable stateless signature generation, and the aggregation server successfully identified weak passwords using privacy-preserving frequency analysis.

The threshold signature protocol and all underlying cryptographic operations, including pairing-based verification and Lagrange interpolation, were proven 100% accurate. The client correctly enforced the 3-of-5 threshold, successfully reconstructing signatures when the threshold was met and rejecting scenarios with insufficient servers.

Security validation confirmed the system's resilience, as security is maintained even if fewer than the threshold of three servers are compromised. Privacy was also validated: servers never observe passwords due to blinding, and the aggregation mechanism protects unique passwords.

The system demonstrated high effectiveness in detecting common weak passwords while maintaining perfect privacy for strong, unique ones. These results collectively validate this distributed approach as a practical, secure, and privacy-preserving advancement over traditional authentication mechanisms, proving its viability for real-world deployment.

CHAPTER 6

CONCLUSIONS AND FUTURE WORK

6.1 CONCLUSIONS

This project has successfully detailed the implementation of an efficient, privacy-preserving scheme for weak password collection in an Internet of Things (IoT) environment. The widespread use of weak and popular passwords in IoT devices represents a critical, large-scale security vulnerability. This project's implementation provides a practical solution that directly addresses this problem by enabling a service provider to build a ”blocklist” of weak passwords without violating user privacy.

The core strength of this system lies in its balance of high security and low computational overhead. By leveraging lightweight cryptographic primitives, the scheme remains practical for resource-constrained devices like the ESP32.

The key achievements and security guarantees of this implementation include: Through password-locked encryption with conditional decryption, the Service Provider can only decrypt a password's hash if and only if it is ”popular” (i.e., submitted by κ or more devices). All unpopular passwords remain encrypted and private. The servers-aided password hardening protocol, implemented using threshold BLS signatures, ensures that a semi-trusted Service Provider cannot pre-compute a dictionary of tags and ciphertexts. This thwarts offline dictionary attacks, which are the primary threat against low-entropy passwords. The use of n Key Servers with a (t,n) threshold means the system does not have a single point of failure. An attacker must compromise at least t servers to break the hardening protocol, making the system far more resilient. The Key Share Renewal algorithm provides a crucial defense against ”perpetual leakage attacks,” where an attacker might slowly compromise servers over a long period. By periodically refreshing the secret shares, all previously stolen shares are rendered useless, ensuring long-term security.

By developing a multi-component architecture (Kotlin/MicroPython IoT clients, C++ Key Servers, and a Java Spring Boot Service Provider), this project provides a robust and scalable blueprint for a real-world deployment. It successfully solves the challenge of weak password collection by making privacy and efficiency the default.

6.2 FUTURE WORK

The following implementations can be done in future. First, develop a dynamic Thresholding with Machine Learning system, where the collection threshold κ adapts based on submission velocity and patterns to distinguish attacks from genuine popularity. Second, conduct large-Scale Performance and Load Testing with millions of simulated IoT devices to identify system bottlenecks. Third, pursue hardware-Specific Cryptographic Optimization by rewriting critical cryptographic routines in C for the ESP32.

REFERENCES

[1] C. Jiang, C. Xu, X. Dong, K. Chen, and G. Yang. An efficient privacy-preserving scheme for weak password collection in internet of things against perpetual leakage. *IEEE Transactions on Information Forensics and Security*, 20:1405–1420, 2025.

[2] J. Zhang, F. Zhang, and X. Huang. Theory and applications of sequentially threshold public-key cryptography: Practical private key safeguarding and secure use for individual users. *IEEE Transactions on Information Forensics and Security*, 20:3220–3233, 2025.

[3] C. Jiang, C. Xu, G. Yang, Z. Zhang, and J. Chen. Device-enhanced password-based threshold single-sign-on authentication. *IEEE Transactions on Information Forensics and Security*, 20:2006–2021, 2025.

[4] C. Jiang, C. Xu, Z. Liu, X. Dong, and W. Zhang. Threshold password-hardening updatable oblivious key management. *IEEE Transactions on Information Forensics and Security*, 20:4799–4814, 2025.

[5] J. Zhang, X. Wang, J. Cui, R. Li, and H. Zhong. A decentralized threshold credential management with fine-grained authentication for vanets. *IEEE Transactions on Information Forensics and Security*, 20:5818–583, 2025.

[6] P.-L. Chiu and K.-H. Lee. Threshold visual cryptography schemes with tagged shares. *IEEE Access*, 8:111330–111346, 2020.

[7] S. J. Shyu. Threshold visual cryptographic scheme with meaningful shares. *IEEE Signal Processing Letters*, 21(12):1521–1525, December 2014.